

Relatório Técnico: Estudo de Caso

ALUNO: FRANCISCO WAGNER FERNANDES DE SOUSA

Fork: <https://github.com/HDB-ATIVIDADES/Task-Manager-using-Flask/>

Etapa 1: Planejamento e Requisitos

Análise de Requisitos

Sistema Flask de gerenciamento de tarefas com autenticação obrigatória, log via syslog e isolamento de dados entre usuários.

Requisitos Funcionais: RF-01 a RF-09 — autenticação, CRUD de tarefas, logs de segurança.

Requisitos Não Funcionais: RNF-01 a RNF-08 — hash de senhas (bcrypt), ORM parametrizado (SQLAlchemy), container Docker, logs centralizados via syslog.

Casos de Uso

ID	Operação	Pré-condição
UC-01	Autenticar (login/logout)	Não autenticado
UC-02	Criar tarefa	Autenticado
UC-03	Visualizar tarefas	Autenticado
UC-04	Editar tarefa	Autenticado + dono
UC-05	Excluir tarefa	Autenticado + dono
UC-06	Pesquisar tarefas	Autenticado

Matriz de Ameaças

ID	Ameaça	Impacto	Mitigação
AM-01	Acesso não autorizado a rotas	Alto	before_request global
AM-02	Força bruta no login	Alto	Alerta Grafana (rate limit não implementado)
AM-03	DoS	Alto	Planejado, não implementado
AM-04	Vazamento de dados	Alto	bcrypt, ORM parametrizado
AM-05	XSS	Médio	Sanitização + escaping Jinja2
AM-06	SQL Injection	Alto	SQLAlchemy queries parametrizadas
AM-07	Escalação horizontal	Alto	Verificação de ownership
AM-08	Vazamento em logs	Médio	Sem dados sensíveis em logs

Etapa 2: Containerização e Modificações de Segurança

Containerização

Ferramentas: Docker 29.5.3, imagem `python:3.9-slim`, Docker Compose.

Sem Dockerfile — código montado como volume, dependências instaladas no `command` do Compose.

```
services:
  app:
    image: python:3.9-slim
```

```
ports: ["5000:5000"]
volumes:
  - ./app
  - /dev/log:/dev/log
working_dir: /app/todo_project
command: sh -c "pip install --no-cache-dir -r /app/requirements.txt && python run.py"
```

Alterações no requirements.txt:

- Flask 2.3.2 → 1.1.4 (compatível com Flask-Login/Bcrypt/WTF)
- SQLAlchemy <2.0 (incompatível com Flask-SQLAlchemy 2.4.1)
- MarkupSafe ==2.0.1 (soft_unicode removido no 2.1+)

Modificações de Segurança

Autenticação global via before_request

```
@app.before_request
def require_auth():
    public_routes = ['login', 'register', 'logout', 'static']
    if request.endpoint not in public_routes and not current_user.is_authenticated:
        app.logger.warning(f'UNAUTHORIZED_ACCESS ip={request.remote_addr}...')
        return redirect(url_for('login'))
```

Syslog Handler conectado a /dev/log com fallback silencioso:

```
syslog_handler = logging.handlers.SysLogHandler(address='/dev/log')
app.logger.addHandler(syslog_handler)
```

Verificação de dono da tarefa

```
if task.author != current_user:
    app.logger.warning(f'SECURITY_VIOLATION...')
    abort(403)
```

Resultados

Modificação	Efeito
before_request global	Toda rota protegida, exceto login/register/logout/static
Syslog	Eventos de segurança enviados para syslog do host
Ownership check	Usuário A não pode editar/excluir tarefa do usuário B
8 novos testes	Validação automatizada das regras de segurança

Lições aprendidas: A migração de @login_required (decorator por rota) para before_request (middleware global) simplificou a manutenção e eliminou o risco de esquecer um decorator em rota nova.

Etapa 3: Testes e Pipeline CI

Testes Automatizados

Ferramentas: pytest 8.4.2, SQLite :memory:, CSRF desabilitado em testes.

```
tests/
+-- conftest.py      # Fixtures (client, auth_client, user, user_with_tasks)
+-- test_forms.py    # 9 testes -- validacao de formularios
+-- test_models.py   # 8 testes -- User, Task, relacionamentos
+-- test_routes.py   # 26 testes -- auth, CRUD, isolamento, syslog, conta
```

Resultado: 43 passed, 1 warning (FSADeprecationWarning) em 10.28s.

Classes de teste:

Classe	Testes	O que valida
TestRegistration	4	Registro, duplicidade
TestLogin	3	Login sucesso/falha, logout
TestAuthorization	5	Redirecionamento sem auth
TestTaskCRUD	4	Criar, listar, editar, excluir
TestTaskIsolation	2	Owner check (403)
TestAccount	3	Página conta, alterar senha
TestSyslog	4	LOGIN_SUCCESS, LOGIN_FAILURE, REGISTER_SUCCESS, OPERATION_SUCCESS
TestForms	9	Validação dos formulários
TestModels	8	Modelos User e Task

Pipeline CI

Ferramenta: GitHub Actions.

Jobs iniciais: build → test.

Job	Descrição	Timeout
build	Checkout + Python 3.9 + cache pip + validação	10 min
test	Instala dependências + 43 testes + junit report	10 min

Lições aprendidas: O cache de dependências pip reduziu o tempo de instalação de ~2 min para ~10s nos jobs subsequentes.

Etapa 4: Análise Estática de Segurança (SAST)

Ferramentas: Bandit (código) e pip-audit (dependências).

Bandit

```
Total issues: 90
HIGH: 0 [OK] (pipeline passa)
MEDIUM: 1 - B104 (SECRET_KEY hardcoded em __init__.py:11)
LOW: 89 - B101 (assert em testes), B105/B106 (strings hardcoded)
```

Threshold: pipeline falha apenas em HIGH. Como não há nenhum, o job passa.

pip-audit

16 vulnerabilidades conhecidas em 4 pacotes — **todas aceitas como risco** por compatibilidade com Flask 1.1.4.

Pacote	Versão	Vulns	Fixação
Flask	1.1.4	2	2.2.5+
Jinja2	2.11.3	4	3.1.3+
Werkzeug	1.0.1	9	2.2.3+
pytest	7.4.4	1	9.0.3

Atualizar qualquer um desses pacotes quebraria a aplicação devido a incompatibilidades na cadeia de dependências.

Lições aprendidas: O Bandit B104 (SECRET_KEY hardcoded) é um lembrete de que configurações sensíveis devem vir de variáveis de ambiente, não do código-fonte. As 16 vulns do pip-audit demonstram o trade-off entre estabilidade funcional e segurança de dependências.

Etapa 5: Análise Dinâmica de Segurança (DAST)

Ferramenta: OWASP ZAP (baseline scan), imagem `ghcr.io/zaproxy/zaproxy:stable`.

Modo headless, Docker Compose na mesma rede, regras de falso positivo em `.zap/rules.tsv`.

Resultados

Métrica	Valor
Alvos	1 (<code>http://app:5000</code>)
Duração	~3 min
Total alertas	16 tipos
HIGH	0
MEDIUM	3
LOW	7
INFORMACIONAL	6

Alertas MEDIUM

ID	Alerta	Ocorrências
10038	Content Security Policy (CSP) Header Not Set	4
10020	Missing Anti-clickjacking Header	4
10009	Vulnerable JS Library (Bootstrap)	1

Alertas LOW (amostra)

ID	Alerta	Ocorrências
10054	Cookie without SameSite Attribute	5
90004	Cross-Origin-Embedder-Policy Missing	4
10036	Server Leaks Version via “Server” Header	5
10021	X-Content-Type-Options Header Missing	5

Nenhum alerta HIGH foi encontrado. Os alertas MEDIUM e LOW são comportamentos esperados em ambiente HTTP de desenvolvimento e em aplicação Flask sem hardening de headers.

Lições aprendidas: O ZAP baseline scan é eficaz para detectar configurações incorretas de segurança (headers ausentes, informações vazadas). Em produção, seria necessário adicionar HTTPS e headers de segurança (CSP, HSTS, X-Frame-Options).

Etapa 6: Entrega Contínua (CD)

Ferramenta: GitHub Actions + GitFlow.

GitFlow

Branch	Uso	Gatilho
<code>feature/*</code>	Desenvolvimento	PR → <code>develop</code>
<code>develop</code>	Integração	Push (CI) / PR (CI + review)
<code>staging</code>	Pré-produção	Push (CI + deploy + DAST)
<code>master</code>	Produção	Push (CI)

Jobs de CD

Job	Gatilho	Depende	Ação
deploy-review	PR → develop	dast	Smoke test + comenta no PR
deploy-staging	Push staging	dast	Deploy + smoke + aprovação manual
dast-staging	Push staging	deploy-staging	ZAP scan no staging

Aprovação manual: Environment `staging` configurado no GitHub com `required reviewers`, garantindo que ninguém faça deploy sem revisão.

Smoke Test

Script Python com `urllib` (sem dependências externas) que percorre: GET `/login` → POST `/register` → POST `/login` → POST `/add_task` → GET `/all_tasks` → GET `/logout`.

Lições aprendidas: A extração de CSRF token do formulário HTML foi necessária porque o Flask-WTF protege todos os formulários por padrão. O smoke test demonstrou que o CD precisa de validação funcional, não apenas de build.

Etapa 7: Monitoramento (Promtail + Loki + Grafana)

Ferramentas: Promtail 3.4.2, Loki 3.4.2, Grafana 11.4.0.

Arquitetura

```

flowchart LR
    A[Flask App<br/>FileHandler] --> B[logs/app.log]
    B --> C[Promtail]
    C --> D[Loki]
    D --> E[Grafana]

```

Serviços no Docker Compose

Serviço	Imagem	Porta
loki	<code>grafana/loki:3.4.2</code>	3100
promtail	<code>grafana/promtail:3.4.2</code>	—
grafana	<code>grafana/grafana:11.4.0</code>	3000

O Flask escreve logs simultaneamente para syslog (`/dev/log`) e para arquivo (`logs/app.log` via `FileHandler`). O Promtail faz tail deste arquivo e envia ao Loki.

Dashboard — 7 Painéis

Painel	Tipo	Finalidade
Timeline de Logs	Logs	Visualização bruta
LOGIN_FAILURE/min	Time series	Taxa de falhas
LOGIN_SUCCESS/min	Time series	Taxa de sucessos
Tasks Criadas (24h)	Stat	Volume de operações
Acessos Não Autorizados	Stat	Violações
Eventos por Severidade	Pie chart	Distribuição
Falhas de Login (threshold 5)	Time series	Alerta brute-force

Alerta de Brute-force

```
count_over_time({job="flask-app"} |= "LOGIN_FAILURE" [1m]) > 5
```

Gatilho: mais de 5 falhas de login no último minuto. Provisionado automaticamente via `config/grafana/alerting/rules.yml`.

Lições aprendidas: A stack Promtail+Loki+Grafana consumiu ~2 GB RAM total, muito mais leve que ELK Stack. A coleta de logs via FileHandler (arquivo) foi mais simples de configurar que syslog remoto. O LogQL para alertas de brute-force mostrou-se eficaz e de fácil manutenção.

Pipeline CI/CD Final

Arquivo `.github/workflows/ci.yml` — 8 jobs: build, test, sast, dast, deploy-review, deploy-staging, dast-staging, monitoring.

```
name: CI/CD
on:
  push: {branches: [develop, staging, master]}
  pull_request: {branches: [develop]}
  workflow_dispatch:
concurrency:
  group: ${github.workflow}-${github.ref}
  cancel-in-progress: true
env:
  PYTHON_VERSION: '3.9'

jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: {python-version: '3.9'}
      - uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: ${runner.os}-pip-${hashFiles('requirements.txt')}
          restore-keys: ${runner.os}-pip-
      - run: pip install --no-cache-dir -r requirements.txt
      - run: cd todo_project && python -c "from todo_project import app; print('OK')"
```

```
test:
  runs-on: ubuntu-latest
  timeout-minutes: 10
  needs: build
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: {python-version: '3.9'}
    - uses: actions/cache@v4
      with:
        path: ~/.cache/pip
        key: ${runner.os}-pip-${hashFiles('requirements.txt')}
        restore-keys: ${runner.os}-pip-
    - run: pip install --no-cache-dir -r requirements.txt
    - run: cd todo_project && python -m pytest tests/ -v --junitxml=../report.xml
    - if: always()
      uses: actions/upload-artifact@v4
      with: {name: test-report, path: report.xml}
```

```
sast:
  runs-on: ubuntu-latest
  timeout-minutes: 10
  needs: build
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-python@v5
      with: {python-version: '3.9'}
    - uses: actions/cache@v4
      with:
```

```

    path: ~/.cache/pip
    key: ${runner.os}-pip-${hashFiles('requirements.txt')}
    restore-keys: ${runner.os}-pip-
- run: pip install -r requirements.txt && pip install bandit pip-audit
- run: cd todo_project && bandit -r . -f json -o ../bandit-report.json --exit-zero
- run: |
    python -c "
    import json
    with open('bandit-report.json') as f:
        data = json.load(f)
        high = [r for r in data.get('results', []) if r['issue_severity'] == 'HIGH']
        if high: exit(1)
    "
- run: pip-audit -r requirements.txt -f json --desc | tee pip-audit-report.json
- if: always()
  uses: actions/upload-artifact@v4
  with: {name: bandit-report, path: bandit-report.json}
- if: always()
  uses: actions/upload-artifact@v4
  with: {name: pip-audit-report, path: pip-audit-report.json}

dast:
runs-on: ubuntu-latest
timeout-minutes: 15
needs: sast
steps:
- uses: actions/checkout@v4
- run: docker compose up -d
- run: |
    for i in $(seq 15); do
        curl -s -o /dev/null http://localhost:5000/login && exit 0
        sleep 2
    done; echo "App failed"; exit 1
- run: |
    mkdir -p zap_report
    NETWORK=$(docker inspect "$(docker compose ps -q app)"
    --format '{{range .NetworkSettings.Networks}}{{.}}{{end}}')
    docker run --rm --network "$NETWORK" \
    -v "$PWD/.zap/rules.tsv":/zap/rules.tsv:ro \
    -v "$PWD/zap_report":/zap/wrk --workdir /zap/wrk \
    ghcr.io/zaproxy/zaproxy:stable /zap/zap-baseline.py \
    -t http://app:5000 -c /zap/rules.tsv \
    -J zap_report.json -r zap_report.html -I
- if: always()
  uses: actions/upload-artifact@v4
  with: {name: zap-report, path: zap_report/}

deploy-review:
if: github.event_name == 'pull_request' && github.base_ref == 'develop'
runs-on: ubuntu-latest
needs: dast
environment: {name: review, url: http://localhost:5000}
steps:
- uses: actions/checkout@v4
- run: docker compose up -d
- run: |
    for i in $(seq 15); do
        curl -s -o /dev/null http://localhost:5000/login && exit 0
        sleep 2
    done; echo "Failed"; exit 1
- run: python scripts/smoke_test.py http://localhost:5000
- uses: actions/github-script@v7
  with:
    script: |
        github.rest.issues.createComment({
            issue_number: context.issue.number,
            owner: context.repo.owner, repo: context.repo.repo,
            body: '## Review Environment Ready - localhost:5000'
        })

deploy-staging:
if: github.event_name == 'push' && github.ref == 'refs/heads/staging'

```

```

runs-on: ubuntu-latest
needs: dast
environment: {name: staging, url: http://localhost:5000}
steps:
  - uses: actions/checkout@v4
  - run: docker compose -p staging up -d
  - run: |
      for i in $(seq 15); do
        curl -s -o /dev/null http://localhost:5000/login && exit 0
        sleep 2
      done; echo "Failed"; exit 1
  - run: python scripts/smoke_test.py http://localhost:5000

dast-staging:
if: github.event_name == 'push' && github.ref == 'refs/heads/staging'
runs-on: ubuntu-latest
timeout-minutes: 15
needs: deploy-staging
steps:
  - uses: actions/checkout@v4
  - run: docker compose -p staging up -d
  - run: |
      for i in $(seq 15); do
        curl -s -o /dev/null http://localhost:5000/login && exit 0
        sleep 2
      done; echo "Failed"; exit 1
  - run: |
      mkdir -p zap_staging_report
      NETWORK=$(docker inspect "$$(docker compose -p staging ps -q app)"
        --format '{{range .NetworkSettings.Networks}}{{.}}{{end}}')
      docker run --rm --network "$NETWORK" \
        -v "$PWD/.zap/rules.tsv":/zap/rules.tsv:ro \
        -v "$PWD/zap_staging_report":/zap/wrk --workdir /zap/wrk \
        ghcr.io/zaproxy/zaproxy:stable /zap/zap-baseline.py \
        -t http://app:5000 -c /zap/rules.tsv \
        -J zap_report.json -r zap_report.html -I
  - if: always()
    uses: actions/upload-artifact@v4
    with: {name: zap-staging-report, path: zap_staging_report/}

monitoring:
if: github.event_name == 'push' && github.ref == 'refs/heads/staging'
runs-on: ubuntu-latest
timeout-minutes: 15
needs: dast-staging
steps:
  - uses: actions/checkout@v4
  - run: docker compose -p staging up -d
  - run: |
      for i in $(seq 30); do
        APP=$(curl -s -o /dev/null -w '%{http_code}' http://localhost:5000/login)
        GRA=$(curl -s -o /dev/null -w '%{http_code}' http://localhost:3000/api/health)
        [ "$APP" = 200 ] && [ "$GRA" = 200 ] && exit 0
        sleep 5
      done; echo "Failed"; exit 1
  - run: python scripts/generate_traffic.py http://localhost:5000
  - run: |
      curl -s "http://localhost:3100/loki/api/v1/query_range" \
        --data-urlencode 'query={job="flask-app"}' | jq '.data.result | length'
  - run: |
      R=$(curl -s "http://localhost:3100/loki/api/v1/query_range" \
        --data-urlencode 'query={job="flask-app"} |="LOGIN_FAILURE"'
        [ "$(echo $R | jq '.data.result[0].values | length // 0')" -ge 5 ]
  - run: curl -s "http://admin:admin@localhost:3000/api/dashboards/uid/app-logs" -o md.json
  - if: always()
    uses: actions/upload-artifact@v4
    with: {name: monitoring-report, path: monitoring-dashboard.json}

```


Considerações Finais

Riscos Aceitos

16 vulnerabilidades de dependências foram **aceitas como risco** devido à incompatibilidade do Flask 1.1.4 com versões corrigidas dos pacotes. A atualização quebraria a aplicação. Recomenda-se um refactor para Flask 2.x+ em versão futura.

Pendências Técnicas

Item	Impacto	Ação Recomendada
SECRET_KEY hardcoded	Baixo (ambiente dev)	Migrar para variável de ambiente
Rate limiting (RNF-03)	Médio	Implementar bloqueio temporário após N tentativas de login
Debug mode ativado	Médio	Desabilitar em staging/produção
HTTPS	Alto	Configurar certificado para staging/produção

Ferramentas Utilizadas (visão geral)

Categoria	Ferramentas
Containerização	Docker, Docker Compose
Testes	pytest
SAST	Bandit, pip-audit
DAST	OWASP ZAP
CI/CD	GitHub Actions
Monitoramento	Promtail, Loki, Grafana
Controle de versão	Git, GitFlow